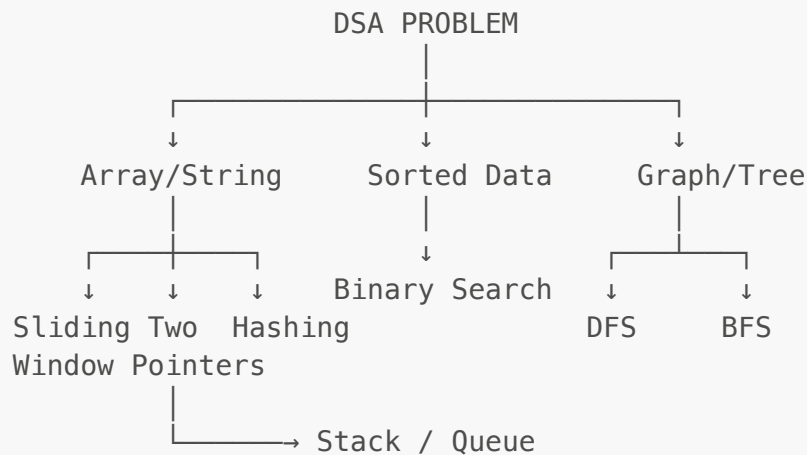


Absolutely. Let's do **DAY 1 — DSA Patterns** in the same detailed style as the Python parts: **mental model** → **when to use** → **pattern** → **Python template** → **example** → **complexity** → **interview traps** → **cheat sheet**.



DAY 1 — DSA PATTERNS

These 6 patterns are much more important than memorizing hundreds of individual problems.



The goal is to look at a problem and think:

"Which pattern does this problem belong to?"



1 SLIDING WINDOW ★★★★★



Core idea

Sliding Window is used when the problem asks about a **contiguous portion** of an array/string.

Keywords:

subarray
substring
contiguous
consecutive
longest
shortest
maximum
minimum
window of size k
at most K
at least K

Instead of repeatedly calculating every possible subarray:

```
[1, 2, 3, 4, 5]
```

```
↑——↑
```

```
window
```

```
↑——↑
```

```
window
```

you maintain a window and move it.

Example: Maximum sum of k elements

```
nums = [2, 1, 5, 1, 3, 2]
k = 3
```

Possible windows:

```
[2, 1, 5] = 8
[1, 5, 1] = 7
[5, 1, 3] = 9 ← maximum
[1, 3, 2] = 6
```

Brute force

You could calculate every window from scratch:

```
for i in range(len(nums) - k + 1):
    total = sum(nums[i:i+k])
```

Potentially $O(nk)$.

Sliding Window

```
def max_sum(nums, k):
    window_sum = sum(nums[:k])
    maximum = window_sum

    for right in range(k, len(nums)):
        window_sum += nums[right]
        window_sum -= nums[right - k]
```

```
        maximum = max(maximum, window_sum)

    return maximum
```

What happens?

[2, 1, 5] → 8

remove 2
add 1

[1, 5, 1] → 7

remove 1
add 3

[5, 1, 3] → 9

We don't recalculate the whole window.

Complexity

```
Time   = O(n)
Space  = O(1)
```

Instead of:

```
O(n × k)
```

Variable Sliding Window ★★★★★

This is more important for interviews.

Example:

Find the longest substring without repeating characters.

```
def longest_unique(s):
    seen = set()
    left = 0
```

```

maximum = 0

for right in range(len(s)):
    while s[right] in seen:
        seen.remove(s[left])
        left += 1

    seen.add(s[right])
    maximum = max(maximum, right - left + 1)

return maximum

```

Mental model:

```

left
↓
[a b c a b]
      ↑
      right

duplicate 'a'
      ↓
move left →

[b c a]

```

The window expands and contracts.

Sliding Window Template

```

left = 0

for right in range(len(nums)):

    # add nums[right]

    while window_is_invalid:
        # remove nums[left]
        left += 1

    # update answer

```

Remember

Sliding Window = contiguous range + maintain window state

2 TWO POINTERS ★★★★★

Two Pointers means using two indexes instead of nested loops.

```
left           right
  ↓             ↓
[1, 2, 3, 4, 5, 6, 7]
```

They can move:

```
→ ←
```

or:

```
→ →
```

depending on the problem.

Example: Two Sum in Sorted Array

```
nums = [1, 2, 3, 4, 6]
target = 6
```

Start:

```
L           R
  ↓         ↓
1  2  3  4  6
```

```
1 + 6 = 7
```

Too large.

Move right:

L					R
↓					↓
1	2	3	4	6	

Actually:

$1 + 6 > 6$
→ move right left

Now:

L				R
↓				↓
1	2	3	4	6

$1 + 4 = 5$

Too small → move left right.

Eventually:

$2 + 4 = 6$

Code

```
def two_sum_sorted(nums, target):
    left = 0
    right = len(nums) - 1

    while left < right:
        total = nums[left] + nums[right]

        if total == target:
            return [left, right]

        if total < target:
            left += 1
        else:
            right -= 1

    return []
```

Complexity

```
Time   = O(n)
Space  = O(1)
```

When to use Two Pointers

Look for:

```
sorted array
pair
two elements
remove duplicates
palindrome
reverse
partition
opposite ends
```

Two Pointers vs Sliding Window

Very common interview confusion.

Sliding Window

```
left  —————  right
      WINDOW
```

Usually concerned with a **contiguous range**.

Two Pointers

```
left →           ← right
```

Two positions move according to some condition.

They can overlap conceptually, but they're not identical.

Example: Palindrome

```
def is_palindrome(s):
    left = 0
    right = len(s) - 1

    while left < right:
        if s[left] != s[right]:
            return False

        left += 1
        right -= 1

    return True
```

```
H E L L E H
↑           ↑
L           R

    ↑       ↑
    L       R

        ↑ ↑
```

3 HASHING

Hashing is about:

Trading memory for fast lookup.

Python gives you:

```
dict
set
```

Typical lookup:

```
Average:
O(1)
```

instead of repeatedly scanning:

$O(n)$

Example: Two Sum

```
nums = [2, 7, 11, 15]
target = 9
```

Brute force:

```
for i in range(n):
    for j in range(i + 1, n):
        ...
```

Complexity:

 $O(n^2)$

Hash Map Solution

```
def two_sum(nums, target):
    seen = {}

    for i, num in enumerate(nums):
        complement = target - num

        if complement in seen:
            return [seen[complement], i]

        seen[num] = i

    return []
```

Mental model:

```
num = 7
target = 9

complement = 9 - 7
```

= 2

Have I seen 2?

YES → answer

Hashing Pattern

Current value

↓

Calculate what you need

↓

Look it up in dict/set

↓

Found?

- └ YES → answer/update
- └ NO → store current

Frequency Counting

Extremely common.

```
from collections import Counter

counts = Counter("banana")
```

Result conceptually:

```
b → 1
a → 3
n → 2
```

Without Counter:

```
counts = {}

for char in "banana":
    counts[char] = counts.get(char, 0) + 1
```

When to use Hashing

Think hashing when you see:

```
frequency  
count  
duplicate  
unique  
lookup  
pair  
complement  
visited  
group  
anagram  
membership
```

Hashing Complexity

Typical:

```
dict lookup → O(1) average  
set lookup  → O(1) average  
insert      → O(1) average  
delete      → O(1) average
```

Worst-case behavior can differ, but interview answers generally use **average O(1)**.

4 BINARY SEARCH ★★★★★

Binary Search means:

Repeatedly cut the search space in half.

The classic requirement:

```
SORTED DATA
```

Example:

```
[-1, 0, 3, 5, 9, 12]
      ↑
    target
```

Instead of checking:

```
-1
0
3
5
...
```

we check the middle.

Example

```
nums = [-1, 0, 3, 5, 9, 12]
target = 9
```

```
low           high
  ↓           ↓
[-1, 0, 3, 5, 9, 12]
      ↑
    mid
```

$3 < 9$

Therefore:

Discard left half

Now:

```
      low      high
      ↓       ↓
[-1, 0, 3, 5, 9, 12]
              ↑
```

Continue.

Code

```
def binary_search(nums, target):  
    left = 0  
    right = len(nums) - 1  
  
    while left <= right:  
        mid = left + (right - left) // 2  
  
        if nums[mid] == target:  
            return mid  
  
        elif nums[mid] < target:  
            left = mid + 1  
  
        else:  
            right = mid - 1  
  
    return -1
```

Complexity:

```
Time   = O(log n)  
Space = O(1)
```

Why $\log n$?

Suppose:

```
n = 1,000,000
```

Binary search doesn't inspect a million elements.

It repeatedly halves:

```
1,000,000  
500,000  
250,000  
125,000
```

...
1

Only around 20 iterations.

Because:

$$2^{20} \approx 1,048,576$$

Binary Search Beyond Arrays ★★★★★

Senior interviews often use **binary search on the answer**.

Example:

Find minimum capacity needed to ship packages within D days.

You don't necessarily search an array.

You search:

possible answer range

minimum capacity ————— maximum capacity
↓
binary search

Mental trigger:

"Can I check whether an answer X is possible?"

If yes, binary search may apply.

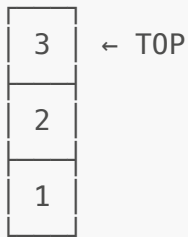
5 STACK & QUEUE ★★★★★

These are about **order of processing**.

Stack = LIFO

Last In, First Out

Think:



Push:

```
push(4)
```

```
4 ← TOP
```

```
3
```

```
2
```

```
1
```

Pop:

```
pop()
```

returns:

```
4
```

Python Stack

Use a list:

```
stack = []  
  
stack.append(10)  
stack.append(20)  
stack.append(30)  
  
value = stack.pop()
```

Result:

30

Operations:

```
append() → O(1) amortized  
pop()    → O(1)
```

Stack Use Cases

Think:

```
parentheses  
undo  
backtracking  
function calls  
DFS  
monotonic stack  
expression evaluation
```

Valid Parentheses

Classic interview problem.

```
def is_valid(s):  
    stack = []  
  
    pairs = {  
        ")": "(",  
        "]": "[",  
        "}": "{"  
    }  
  
    for char in s:  
        if char in "([{":  
            stack.append(char)  
  
        else:  
            if not stack or stack.pop() != pairs[char]:  
                return False
```



```
return not stack
```

Example:

```
{ [ ] }  
  
{ → push  
[ → push  
] → pop [  
} → pop {
```

Valid.

Queue = FIFO

First In, First Out

Think:

FRONT	REAR
↓	↓
[1] [2] [3] [4]	
↑	↑
remove	add

1 enters first → 1 leaves first.

Python Queue

Don't use:

```
list.pop(0)
```

because removing from the front of a list is **O(n)**.

Use:

```
from collections import deque  
  
queue = deque()
```

```
queue.append(1)
queue.append(2)
queue.append(3)

queue.popleft()
```

Result:

1

Typical operations:

```
append() → O(1)
popleft() → O(1)
```

Queue Use Cases

Think:

```
BFS
scheduling
task processing
requests
streams
producer/consumer
```

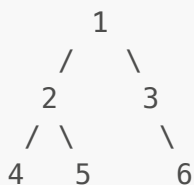
Stack vs Queue

Stack	Queue
LIFO	FIFO
Last enters → first leaves	First enters → first leaves
<code>append()</code>	<code>append()</code>
<code>pop()</code>	<code>popleft()</code>
DFS/backtracking	BFS/scheduling

6 DFS / BFS ★★★★★

These are **graph/tree traversal patterns**.

Suppose:



You need to visit nodes.

Two fundamental approaches:

DFS

↓

Go deep first

BFS

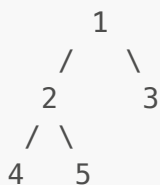
↓

Go level by level

DFS — Depth First Search

DFS says:

Go as deep as possible before coming back.



Possible DFS:

1 → 2 → 4 → 5 → 3

Mental model:

```
1
↓
2
↓
4
↑
2
↓
5
↑
1
↓
3
```

Recursive DFS

```
def dfs(node):
    if node is None:
        return

    print(node.val)

    dfs(node.left)
    dfs(node.right)
```

Recursion itself behaves like a **stack**.

```
DFS
↓
Call Stack
↓
deep
deep
deep
↓
backtrack
```

Iterative DFS

Use a stack:

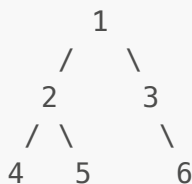
```
def dfs(start, graph):  
    stack = [start]  
    visited = {start}  
  
    while stack:  
        node = stack.pop()  
  
        for neighbor in graph[node]:  
            if neighbor not in visited:  
                visited.add(neighbor)  
                stack.append(neighbor)
```

BFS — Breadth First Search

BFS says:

Visit everything at the current level before going deeper.

Tree:



BFS:

```
Level 0 → 1  
Level 1 → 2, 3  
Level 2 → 4, 5, 6
```

Result:

```
1 → 2 → 3 → 4 → 5 → 6
```

BFS Uses Queue

```
from collections import deque

def bfs(start, graph):
    queue = deque([start])
    visited = {start}

    while queue:
        node = queue.popleft()

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
```

Mental model:

QUEUE

```
[1]
↓
remove 1
↓
add 2, 3

[2, 3]
↓
remove 2
↓
add 4, 5

[3, 4, 5]
...
```

DFS vs BFS

DFS	BFS
Goes deep	Goes level by level
Stack / recursion	Queue
Good for path exploration	Good for shortest unweighted path
Backtracking	Level-order traversal
Connected components	Minimum edges
Cycle detection	Nearest/closest problems



Important BFS Interview Pattern

If the question says:

"Shortest path in an unweighted graph"

Immediately think:

BFS

Why?

Because BFS explores:

```
distance 0
distance 1
distance 2
distance 3
...
```

So the first time you reach a node, you've reached it using the minimum number of edges.



How to Recognize Each Pattern

This is probably the **most useful section for interviews**.

Sliding Window

Question says:

```
longest substring
shortest subarray
maximum sum of k consecutive elements
contiguous
at most K
```

Think:

WINDOW

Two Pointers

Question says:

```
sorted array  
pair  
palindrome  
remove duplicates  
left/right
```

Think:

TWO INDEXES

Hashing

Question says:

```
frequency  
duplicate  
unique  
lookup  
complement  
count  
group
```

Think:

DICT / SET

Binary Search

Question says:

```
sorted  
search  
minimum possible  
maximum possible  
find boundary  
search space
```

Think:

HALF THE SEARCH SPACE

Stack

Question says:

```
parentheses
undo
backtracking
next greater element
nested structure
```

Think:

LIFO

Queue / BFS

Question says:

```
level
shortest path
nearest
first
minimum steps
```

Think:

FIFO / LEVEL ORDER

DFS

Question says:

```
explore
path
connected component
tree traversal
backtracking
island
```

Think:

GO DEEP



Pattern Recognition Table

Problem clue	Pattern
Contiguous subarray	Sliding Window
Contiguous substring	Sliding Window
Fixed window size	Sliding Window
Longest/shortest window	Sliding Window
Sorted array + pair	Two Pointers
Palindrome	Two Pointers
Duplicate detection	Hashing
Frequency counting	Hashing
Fast lookup	Hashing
Sorted search	Binary Search
Search minimum/maximum answer	Binary Search
Matching brackets	Stack
Undo/backtracking	Stack
First-in-first-out	Queue
Level order	BFS
Shortest unweighted path	BFS
Explore all paths	DFS
Connected components	DFS/BFS
Tree traversal	DFS/BFS

Complexity Cheat Sheet

```
Hash Map lookup      → O(1) average
Hash Set lookup      → O(1) average

Sliding Window      → O(n)
Two Pointers         → O(n)

Binary Search        → O(log n)

Stack push/pop       → O(1)
Queue enqueue/dequeue → O(1)
```

```
DFS          →  $O(V + E)$   
BFS          →  $O(V + E)$ 
```

For a tree with n nodes:

```
DFS →  $O(n)$   
BFS →  $O(n)$ 
```



The Most Important Templates

Sliding Window

```
left = 0  
  
for right in range(len(nums)):  
    # add nums[right]  
  
    while invalid:  
        # remove nums[left]  
        left += 1  
  
    # update answer
```

Two Pointers

```
left = 0  
right = len(nums) - 1  
  
while left < right:  
    if condition:  
        left += 1  
    else:  
        right -= 1
```

Hashing

```
seen = {}
```

```
for x in nums:

    if x in seen:
        # found

    seen[x] = ...
```

Binary Search

```
left = 0
right = len(nums) - 1

while left <= right:

    mid = (left + right) // 2

    if nums[mid] == target:
        return mid

    elif nums[mid] < target:
        left = mid + 1

    else:
        right = mid - 1
```

Stack

```
stack = []

for x in data:

    while stack and condition:
        stack.pop()

    stack.append(x)
```

BFS

```
from collections import deque

queue = deque([start])
visited = {start}
```

```
while queue:
    node = queue.popleft()

    for neighbor in graph[node]:
        if neighbor not in visited:
            visited.add(neighbor)
            queue.append(neighbor)
```

DFS

```
def dfs(node):
    if node in visited:
        return

    visited.add(node)

    for neighbor in graph[node]:
        dfs(neighbor)
```



Common Interview Mistakes

✗ Mistake 1: Using nested loops immediately

```
for i in range(n):
    for j in range(n):
```

Before doing this, ask:

Can hashing, two pointers, or sliding window reduce this to $O(n)$?

✗ Mistake 2: Using binary search without a valid property

Binary search requires more than simply "I want something fast."

You need a search space with a property that lets you safely discard half.

✗ Mistake 3: Using `list.pop(0)` as a queue

Bad:

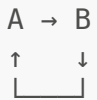
```
queue.pop(0)
```

Prefer:

```
from collections import deque  
  
queue.popleft()
```

✗ Mistake 4: Forgetting **visited**

Graph traversal can loop forever:



Use:

```
visited = set()
```

✗ Mistake 5: Confusing substring with subsequence

```
"abc"
```

Substring:

```
"ab"  
"bc"
```

must be **contiguous**.

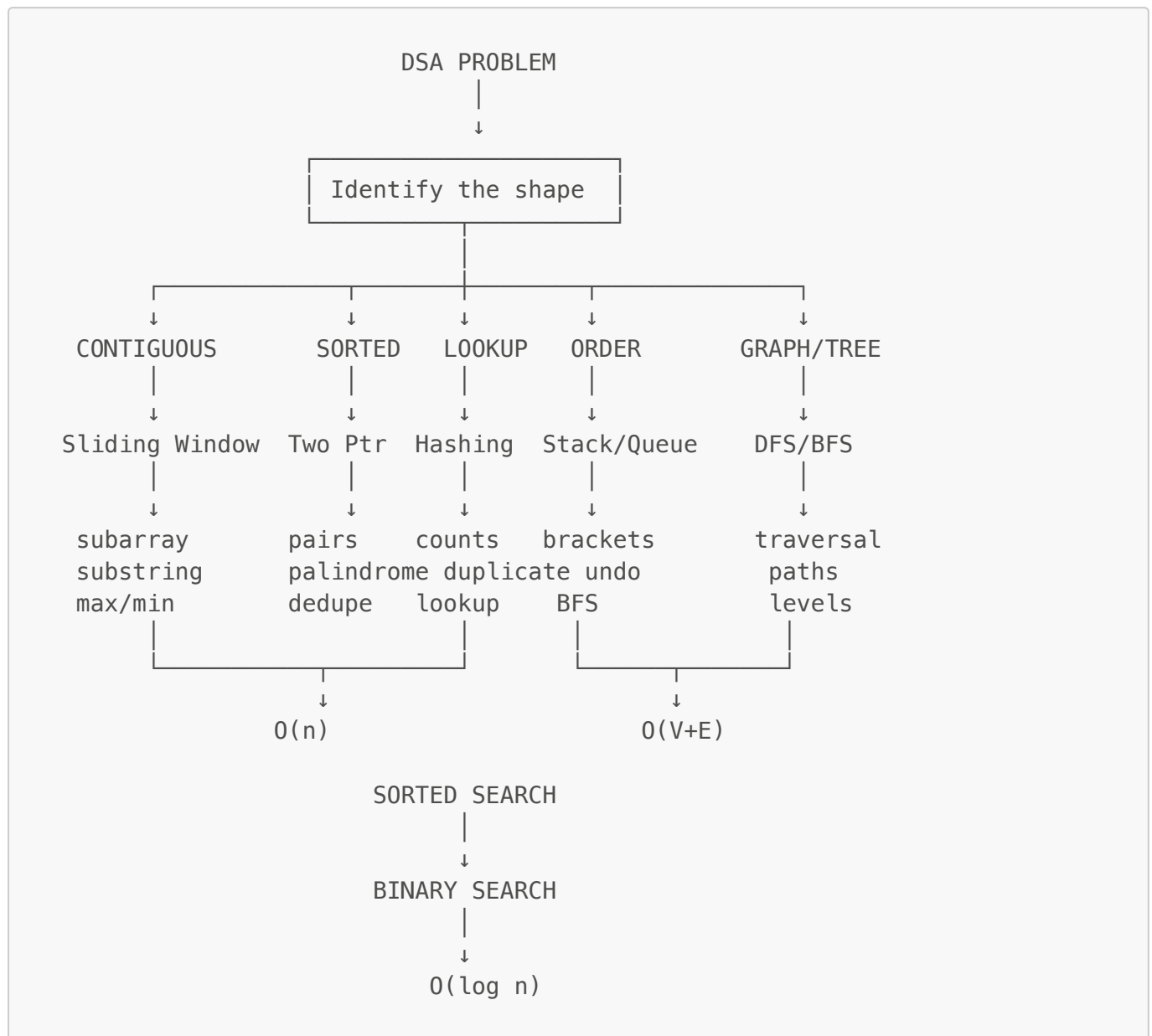
Subsequence:

```
"ac"
```

can skip characters.

This distinction frequently determines whether Sliding Window applies.

FINAL MASTER MAP



DAY 1 — What You Should Master

Don't just memorize the six names.

You should reach this level:

"Longest substring..."

↓

Sliding Window

"Two numbers in sorted array..."

↓
Two Pointers

"Have I seen this before?"
↓
Hashing

"Sorted / monotonic search space..."
↓
Binary Search

"Matching / nested / undo..."
↓
Stack

"Shortest unweighted path / levels..."
↓
BFS

"Explore every path / connected..."
↓
DFS

🔥 The real DSA skill

Problem → recognize pattern → choose data structure → establish invariant → write template → analyze complexity.

That's much more valuable than memorizing individual LeetCode solutions.